**ONLINE EXAMINATION & PROCTORING SYSTEM**

---

## 1. TITLE PAGE

**Course Name:** Object Oriented Programming (OOP)
**Project Title:** Online Examination & Proctoring System
**Student Name:** Manjeet Kumar
**Roll Number:** 27
**Instructor Name:** Dr. Soumadip Biswas
**Semester:** Spring 2026

---

## 2. PROBLEM STATEMENT

With the increasing adoption of online education, conducting secure and fair examinations has become a major challenge. Traditional examination systems rely heavily on physical supervision, which is not feasible in remote environments.

Existing online examination platforms often lack proper monitoring mechanisms, making them vulnerable to cheating practices such as tab switching, inactivity, or unauthorized collaboration. This reduces the credibility of online assessments and affects academic integrity.

To address these issues, this project proposes a **Java-based Online Examination & Proctoring System**. The system allows students to take exams remotely while incorporating basic proctoring features such as inactivity detection and tab-switch alerts. It also automates exam management, question handling, and result evaluation.

This solution aims to provide a **secure, efficient, and scalable platform** for conducting online examinations.

---

## 3. SYSTEM DESIGN

### 3.1 UML Class Diagram (Description)

The system consists of the following main classes:

- **User (Abstract Class)**
- **Student (Derived Class)**
- **Admin (Derived Class)**
- **Question**
- **Exam**
- **ExamTimer**
- **ProctoringSystem**

Relationships:

- Student and Admin inherit from User

- Admin manages Exam
- Exam contains multiple Question objects
- ExamTimer controls exam duration
- ProctoringSystem monitors behavior

---

### 3.2 Description of Major Classes

**User**

- Abstract class
- Defines login and logout methods

**Student**

- Inherits User
- Takes exam and submits answers

**Admin**

- Adds and manages questions

**Question**

- Stores question text, options, and correct answer

**Exam**

- Handles exam logic
- Stores questions and student responses

**ExamTimer**

- Implements thread for countdown timer

**ProctoringSystem**

- Detects suspicious behavior

---

### 3.3 Package Structure

```
src/
  ├── Main.java
  ├── User.java
  ├── Student.java
  ├── Admin.java
  ├── Question.java
```

```
├── Exam.java
├── ExamTimer.java
├── ProctoringSystem.java
├── InvalidInputException.java
```

---

### 3.4 Interaction Flow

1. User logs in
2. Admin adds questions
3. Student starts exam
4. Timer begins
5. Questions displayed
6. Answers recorded
7. Proctoring monitors activity
8. Exam auto-submits
9. Result generated

---

## 4. OOP CONCEPTS DEMONSTRATED

---

### 4.1 Abstraction

**Where Used:** User class
**Why Used:** To define common behavior for all users

**Code Snippet**

```
abstract class User {
    protected String username;
    protected String password;

    public abstract void login(String username, String password);
    public abstract void logout();
}
```

**Explanation**

This abstract class hides implementation details and ensures that all subclasses implement login and logout functionality.

---

### 4.2 Inheritance

**Where Used:** Student and Admin classes
**Why Used:** Code reusability and hierarchy

**Code Snippet**

```
class Student extends User {
    public Student(String username, String password) {
        super(username, password);
    }
}
```

**Explanation**

Student inherits properties and methods from User, reducing redundancy.

---

### 4.3 Polymorphism

**Where Used:** login() method
**Why Used:** Different behavior for different users

**Code Snippet**

```
@Override
public void login(String username, String password) {
    if (this.username.equals(username) && this.password.equals(password)) {
        System.out.println("Login successful");
    }
}
```

**Explanation**

The same method behaves differently depending on whether it is called by Student or Admin.

---

### 4.4 Exception Handling

**Where Used:** Input validation
**Why Used:** Prevent system crashes

**Code Snippet**

```
if (ans < 1 || ans > 4) {
    throw new InvalidInputException("Invalid option");
}
```

**Explanation**

Ensures only valid inputs are accepted during exam.

---

### 4.5 Collections

**Where Used:** Exam class
**Why Used:** Efficient data handling

**Code Snippet**

```
List<Question> questions = new ArrayList<>();

Map<Integer, Integer> answers = new HashMap<>();
```

**Explanation**

ArrayList stores questions and HashMap stores answers dynamically.

---

### 4.6 Multithreading

**Where Used:** ExamTimer
**Why Used:** Run timer parallel to exam

**Code Snippet**

```
class ExamTimer extends Thread {
    public void run() {
        System.out.println("Timer running...");
    }
}
```

**Explanation**

Allows exam timer to run independently without blocking main program.

---

## 5. IMPLEMENTATION HIGHLIGHTS

---

### 5.1 Question Handling

```
public boolean checkAnswer(int answer) {
    return answer == correctAnswer;
```

```
}
```

**Justification:**
Ensures automatic evaluation of answers.

---

### 5.2 Exam Flow

```
for (int i = 0; i < questions.size(); i++) {
    questions.get(i).displayQuestion();
}
```

**Justification:**
Controls sequential display of questions.

---

### 5.3 Result Calculation

```
if (questions.get(i).checkAnswer(studentAnswers.get(i))) {
    score++;
}
```

**Justification:**
Implements automatic scoring logic.

---

### 5.4 Timer Implementation

```
Thread.sleep(1000);
```

**Justification:**
Simulates real-time countdown.

---

### 5.5 Proctoring Alert

```
System.out.println("Warning: Tab switching detected!");
```

**Justification:**
Provides basic monitoring for suspicious behavior.

---

## 6. TESTING & ERROR HANDLING

### Test Cases

- Valid login
- Invalid login
- Correct answers

- Incorrect answers
- Timer completion

**Edge Cases**

- No input
- Invalid option selection
- Time expiry before completion

**Failure Scenarios**

- Wrong credentials → error message
- Invalid input → handled via exception
- Timer interruption → safe termination

---

## 7. GIT WORKFLOW

**Commit History Summary**

- Day 1 → Project setup
- Day 2 → Inheritance
- Day 3 → Question logic
- Day 4 → Exam system
- Day 5 → Timer & proctoring
- Day 6 → Integration
- Day 7 → Exception handling
- Day 8 → Testing

**Development Progression**

The project was developed incrementally:

- Started with structure
- Added core logic
- Enhanced features
- Performed testing and debugging

---

## 8. CONCLUSION & FUTURE SCOPE

**Conclusion**

The project successfully demonstrates the implementation of Object-Oriented Programming concepts in a real-world system. It provides a secure and efficient way to conduct online exams with basic proctoring.

**Future Scope**

- GUI using JavaFX
- Database integration
- AI-based proctoring
- Web-based deployment

---

**APPENDIX**

**A. Project Proposal**

**1.Project Title: Online Examination & Proctoring System (Java Based)**

**2. Problem Statement With the rapid growth of online education, conducting secure examinations has become challenging. Many existing online exam systems lack proper monitoring, which increases the chances of cheating and unfair practices. Educational institutions require a system that can conduct exams remotely while maintaining academic integrity. The proposed Online Examination & Proctoring System provides a secure platform where students can take exams online while being monitored through automated proctoring features. The system will manage user authentication, question management, t imed exams, result processing, and basic suspicious activity detection. This solution helps institutions conduct fair, efficient, and reliable online assessments while reducing manual effort.**

**3. Target User • Schools and Colleges • Universities • Coaching Institutes • Training Organizations**

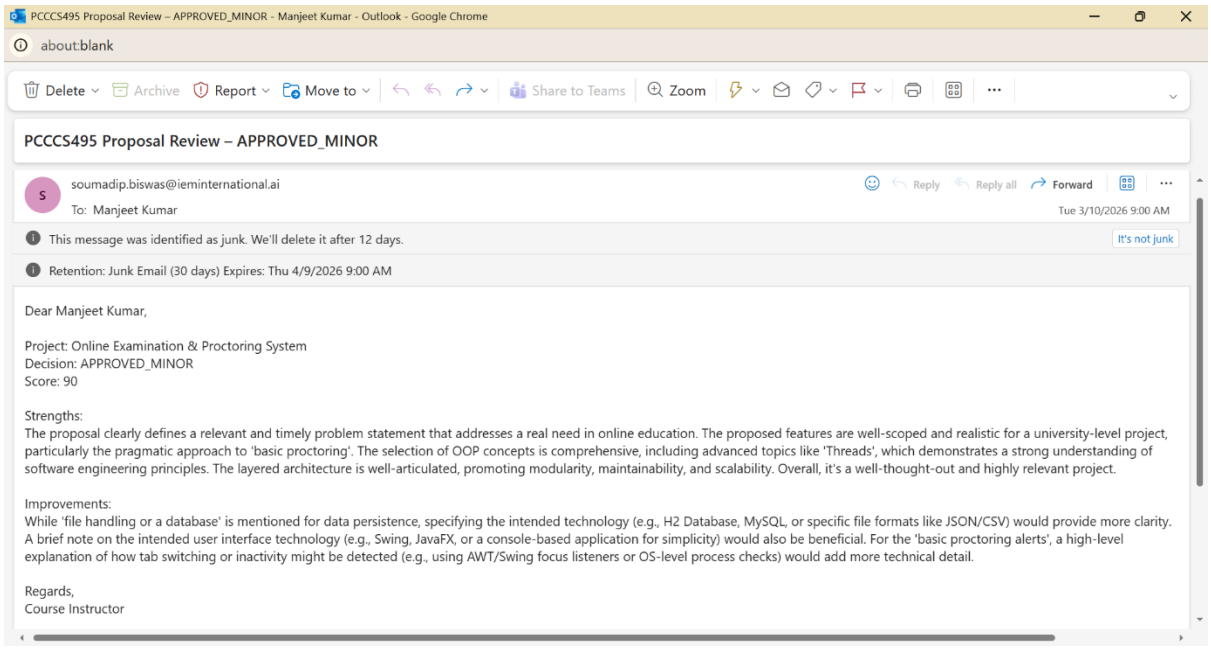
**4. Core Features • Secure login for Admin and Students • Question bank management • Timed online examination • Auto submission after time ends • Basic proctoring alerts (tab switching / inactivity) • Automatic result generation • Admin dashboard for exam monitoring**

**5. OOP Concepts to be Used .Abstraction: Abstract class User defining common functions like login and logout. .Inheritance: Classes such as Student and Admin inherit from User. .Polymorphism: Different implementations of exam functions like startExam() for different exam types. .Exception Handling: Handling invalid login, incorrect inputs, and runtime errors. .Collections / Threads: ArrayList for question storage, HashMap for answers, and threads for exam t imer.**

**6. Proposed Architecture Description The system follows a layered architecture with three main layers: Presentation Layer, Business Logic Layer, and Data Layer. The presentation layer handles user interaction, the business logic layer manages exam operations and proctoring logic, and the data layer stores user details, questions, and results using file**

**handling or a database. This structure ensures modular, secure, and maintainable system design.**

## B. Review Mail



PCCCS495 Proposal Review – APPROVED_MINOR - Manjeet Kumar - Outlook - Google Chrome

about:blank

Delete | Archive | Report | Move to | Share to Teams | Zoom

**PCCCS495 Proposal Review – APPROVED_MINOR**

S soumadip.biswas@ieminternational.ai

To: Manjeet Kumar

Reply | Reply all | Forward

Tue 3/10/2026 9:00 AM

This message was identified as junk. We'll delete it after 12 days. It's not junk

Retention: Junk Email (30 days) Expires: Thu 4/9/2026 9:00 AM

Dear Manjeet Kumar,

Project: Online Examination & Proctoring System
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly defines a relevant and timely problem statement that addresses a real need in online education. The proposed features are well-scoped and realistic for a university-level project, particularly the pragmatic approach to 'basic proctoring'. The selection of OOP concepts is comprehensive, including advanced topics like 'Threads', which demonstrates a strong understanding of software engineering principles. The layered architecture is well-articulated, promoting modularity, maintainability, and scalability. Overall, it's a well-thought-out and highly relevant project.

Improvements:
While 'file handling or a database' is mentioned for data persistence, specifying the intended technology (e.g., H2 Database, MySQL, or specific file formats like JSON/CSV) would provide more clarity. A brief note on the intended user interface technology (e.g., Swing, JavaFX, or a console-based application for simplicity) would also be beneficial. For the 'basic proctoring alerts', a high-level explanation of how tab switching or inactivity might be detected (e.g., using AWT/Swing focus listeners or OS-level process checks) would add more technical detail.

Regards,
Course Instructor